

# **Nettoyage des données — suite**

## Analyse des données — Séance 3, suite

---

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

## Où nous en sommes

La semaine dernière, nous avons vu comment **trouver** les valeurs manquantes, et distingué leurs trois origines possibles.

|                               |                                                |
|-------------------------------|------------------------------------------------|
| Complètement au hasard        | l'absence ne dépend de rien                    |
| Au hasard, conditionnellement | l'absence dépend de variables observées        |
| <b>Non au hasard</b>          | <b>l'absence dépend de la valeur elle-même</b> |

Aujourd'hui : reconnaître le troisième cas, puis **décider quoi faire** — et documenter la décision.

### Le fil de la séance

Le nettoyage n'est pas une opération technique. C'est une suite de décisions, dont chacune modifie le résultat.

## Reconnaître le troisième cas

```
df["nr"] = df["SALAIRE_NET"] == 999999  
100 * df.groupby("DIPLOME")["nr"].mean()
```

| DIPLOME         | 1     | 2    | 3   | 4   | 5   | 6    | 7    | 9    |
|-----------------|-------|------|-----|-----|-----|------|------|------|
|                 | aucun | BEPC | CAP | bac | +2  | +3/4 | +5   | n.r. |
| Non-réponse (%) | 3,3   | 4,4  | 4,9 | 6,3 | 8,0 | 7,7  | 10,6 | 5,9  |

Trois fois plus de non-réponse en haut de l'échelle des diplômes qu'en bas. La non-réponse dépend d'une variable fortement liée au salaire, ce qui rend l'hypothèse de hasard intenable.

La dernière colonne n'est pas un niveau de diplôme. C'est la non-réponse au diplôme, et ses 5,9 % ne se lisent pas sur le gradient. Une modalité « non-réponse » qui apparaît dans un profil est une erreur de nettoyage, pas un résultat.

Sur les variables de revenu et de patrimoine, la non-réponse est presque toujours du troisième type.

- Les très hauts revenus déclarent moins souvent.
- Les revenus non déclarés sont donc, en moyenne, plus élevés.
- Supprimer ces lignes sous-estime la moyenne et les inégalités.

On ne peut pas trancher à partir des données seules : par construction, on ne voit pas ce qui manque.

Ce qu'on peut faire, c'est **le dire**, et mesurer la sensibilité du résultat à l'hypothèse retenue.

## Hors champ n'est pas non-réponse

Trois colonnes du fichier contiennent des cases vides. Aucune des trois ne signifie la même chose.

| Colonne         | Effectif | Ce que le vide signifie                         |
|-----------------|----------|-------------------------------------------------|
| SALAIRE_NET = 0 | 675      | Non-salarié : <b>la question ne se pose pas</b> |
| MOTIF_ABS vide  | 7 153    | N'a pas été absent : <b>sans objet</b>          |
| PRIME vide      | 4 442    | N'a touché <b>aucune</b> prime                  |

Une personne **hors champ** n'aurait pas dû répondre. Une **non-réponse** aurait dû. Les traiter de la même façon est une faute.

Remplacer une prime absente par la prime moyenne invente un versement qui n'a pas eu lieu. Supprimer la ligne élimine précisément les salariés sans prime, c'est-à-dire la majorité.

## La modalité 9 n'est pas toujours une non-réponse

| Code     | MOTIF_ABS — libellé du dictionnaire | Effectif  |
|----------|-------------------------------------|-----------|
| 1        | Congés                              | 458       |
| 2        | Maladie                             | 175       |
| 3        | Congé de maternité ou de paternité  | 34        |
| 4        | Formation                           | 54        |
| 5        | Raisons familiales                  | 45        |
| <b>9</b> | <b>Chômage partiel</b>              | <b>61</b> |
| 99       | Non-réponse                         | 32        |

Ici la non-réponse est codée **99**, et **9** est une vraie situation.

`df["MOTIF_ABS"].replace(9, np.nan)` détruirait soixante et une situations de chômage partiel : sans message d'erreur, et en supprimant exactement la catégorie qu'un économiste du travail cherchait.

## La censure haute

Une valeur qui revient beaucoup trop souvent dans une variable continue signale un plafond.

```
df["HEURES"].value_counts().sort_index().tail(8)
```

|                 |    |    |    |    |    |    |           |
|-----------------|----|----|----|----|----|----|-----------|
| <b>Heures</b>   | 54 | 55 | 56 | 57 | 58 | 59 | <b>60</b> |
| <b>Effectif</b> | 35 | 32 | 35 | 29 | 27 | 35 | 273       |

Le dictionnaire tranche : la modalité se lit « 60 heures ou plus ». Ce n'est pas une erreur, c'est un regroupement décidé par le producteur. Même chose sur SALAIRE\_NET, où les quantiles à 99 % et à 99,9 % valent tous deux 10 000 euros.

Un indice de Gini calculé sur cette colonne est une borne inférieure. La censure comprime le haut de la distribution, là où se concentre l'inégalité mesurée.

**Décider**

---

## Trois options, aucune neutre

| Option                       | Ce qu'on fait                                  | Ce que cela coûte                  |
|------------------------------|------------------------------------------------|------------------------------------|
| Supprimer les lignes         | on retire les observations incomplètes         | échantillon réduit, biais possible |
| Imputer                      | on remplace par une valeur                     | fausse précision                   |
| <b>Conserver et signaler</b> | on ne touche à rien, on rapporte les effectifs | analyse plus lourde                |

Signaler n'est pas une action sur les données : c'est une action sur le compte rendu. On laisse les manquants en place, les calculs les ignorent, et on dit combien d'observations ont réellement servi.

C'est le cas général — et la seule option qui ne suppose rien. Supprimer suppose que les manquants sont sans structure ; imputer suppose qu'on sait par quoi les remplacer.

## Supprimer : ce qui disparaît vraiment

```
avant = len(com)
propre = com.dropna(subset=["TAUX_CHOMAGE", "NB_ENTREPRISES"])
print(avant, "->", len(propre))           # 45 -> 37
```

Huit communes sur quarante-cinq, soit 18 % du fichier. Le vrai problème n'est pas le nombre :

|                    | Conservées | Supprimées |
|--------------------|------------|------------|
| Population moyenne | 156 380    | 1 786      |
| Communes rurales   | 0          | 6          |

### Le contrôle à faire

Comparez les lignes supprimées aux lignes conservées. Ici la catégorie rurale disparaît entièrement : le fichier nettoyé ne décrit plus les communes françaises, mais celles de plus de trois mille habitants.

# Le piège de l'indexation chaînée

```
com[com["POP"] < 2000]["TAUX_CHOMAGE"] = 0
```

```
SettingWithCopyWarning: A value is trying to be set on  
a copy of a slice from a DataFrame
```

Deux crochets successifs produisent une *copie* : la modification s'applique à la copie, puis disparaît. Aucune erreur, aucun effet.

```
com.loc[com["POP"] < 2000, "TAUX_CHOMAGE"] = 0
```

.loc désigne lignes et colonnes en une seule opération. **Pour modifier, c'est toujours .loc.**

## Supprimer une ligne, ou supprimer une colonne

```
df.dropna(subset=["SALAIRE_NET"])    # retire des OBSERVATIONS  
df.drop(columns=["PRIME"])           # retire une VARIABLE
```

Deux opérations, deux pertes de nature différente.

- Retirer des **lignes** réduit l'échantillon, et le déforme si les manquants ne sont pas au hasard.
- Retirer une **colonne** conserve tout l'échantillon, mais vous prive d'une variable.

Ici PRIME est vide pour 55 % des postes, et pourtant le vide y est une information.  
SALAIRE\_NET manque pour 7 % des salariés, ce qui ne justifie pas de perdre 7 % de l'échantillon si votre question porte sur la durée du travail.

La question n'est pas « combien manque-t-il ? » mais « en ai-je besoin ? ».

## Imputer par la moyenne : ce que cela casse

```
df["SALAIRE_NET"].fillna(df["SALAIRE_NET"].mean())
```

Sur les 7 326 postes salariés, dont 524 sans salaire renseigné :

|                             | Conserver | Imputer |
|-----------------------------|-----------|---------|
| Moyenne                     | 2 610     | 2 610   |
| Écart-type                  | 1 465     | 1 412   |
| Corrélation avec le diplôme | 0,356     | 0,342   |
| Observations annoncées      | 6 802     | 7 326   |

La moyenne ne bouge pas d'un centime. C'est ce qui rend l'opération dangereuse : rien ne signale que quelque chose a changé.

Ce qui change, c'est la dispersion, les corrélations, et le nombre d'observations annoncé. Les erreurs-types seront trop petites, et l'inférence trop confiante.

## Ce que nous ferons dans ce cours

1. **Conserver et signaler** : ne rien retirer, et rapporter le nombre d'observations effectivement utilisées pour chaque statistique. C'est la colonne « Observations » de tout tableau d'article.
2. **Documenter** : dire combien de lignes sont concernées et sur quelles variables.
3. **Tester la sensibilité** : refaire le calcul principal sous deux hypothèses, et regarder si la conclusion tient.

Un résultat qui change de signe selon le traitement des manquants n'est pas un résultat. Un résultat qui tient sous les deux hypothèses est solide, et le dire renforce l'argument.

# Écrire votre première fonction

Quatre colonnes de ce fichier codent la non-réponse par un nombre. La règle s'écrit une fois.

```
def nettoyer(serie, codes_manquants):  
    s = pd.to_numeric(serie, errors="coerce")  
    return s.replace(codes_manquants, np.nan)  
  
df["SALAIRE_NET"] = nettoyer(df["SALAIRE_NET"], [999999])  
df["PRIME"]       = nettoyer(df["PRIME"],       [999999])  
df["HEURES"]      = nettoyer(df["HEURES"],      [99])  
df["DIPLOME"]     = nettoyer(df["DIPLOME"],     [9])
```

Trois bénéfices : la règle est écrite une seule fois, elle porte un nom qui dit ce qu'elle fait, et la corriger la corrige partout.

**Notez la ligne absente : MOTIF\_ABS.** La fonction est correcte ; la liste de codes ne l'est jamais par défaut. Elle vient du dictionnaire, colonne par colonne.

## Les doublons

---

## Deux questions différentes

```
df.duplicated().sum()          # 12
df.duplicated(subset=["IDENT"]).sum()  # 632
```

- Douze lignes sont **entièrement identiques** à une autre. C'est presque toujours une erreur : import répété, fusion mal faite.
- Six cent trente-deux **identifiants sont répétés**. Ce n'est pas forcément une erreur. Tout dépend de l'unité statistique.

Rappel de la diapositive d'ouverture : **une ligne est un poste**, pas une personne. Une même personne peut occuper deux postes dans l'année.

La clé du fichier n'est pas IDENT, c'est le couple IDENT + NOPOSTE.

## Regarder avant de supprimer

```
doublons = df[df.duplicated(subset=["IDENT"], keep=False)]  
doublons.sort_values(["IDENT", "NOPOSTE"]).head(6)
```

| IDENT    | NOPOSTE | TEMPS | HEURES | SALAIRE_NET |
|----------|---------|-------|--------|-------------|
| 00006195 | 1       | 1     | 34     | 2 499,21    |
| 00006195 | 2       | 2     | 27     | 560,61      |

Les deux lignes diffèrent sur le temps de travail, les heures et le salaire. Ce sont deux emplois distincts, pas une erreur.

### Le diagnostic, et l'ordre des opérations

On supprime d'abord les douze lignes identiques partout. Il reste 8 000 lignes et 620 identifiants répétés : ceux-là se gardent. Si votre question porte sur les personnes, il faut agréger, pas supprimer.

## Documenter

---

# Ne jamais écraser les données brutes

```
brut = pd.read_csv("cm03-salaires.csv", sep=";")    # jamais modifie  
df    = brut.copy()                                # on travaille ici
```

Deux raisons.

- Vous vous tromperez, et il faudra revenir en arrière sans tout retélécharger.
- La comparaison brut / nettoyé est ce qui vous permet de *mesurer* l'effet de vos décisions.

| Étape                   | Décision    | Lignes | Salaire moyen | Justification               |
|-------------------------|-------------|--------|---------------|-----------------------------|
| Chargement              | —           | 8 012  | 67 745        | Fichier brut                |
| Doublons techniques     | Suppression | 8 000  | 67 719        | Lignes identiques partout   |
| Code 999 999 → manquant | Conversion  | 8 000  | 2 375         | Dictionnaire, SALAIRE_NET   |
| Non-salariés            | Exclusion   | 7 326  | 2 610         | Hors champ : pas de salaire |

Trois observations sur ce tableau.

- Supprimer douze lignes sur huit mille ne change presque rien : 67 745 devient 67 719.
- Traiter un seul code de non-réponse fait passer la moyenne de 67 745 à 2 375. Une ligne de code, un facteur vingt-huit.
- Retirer les non-salariés la fait remonter à 2 610, parce que leurs zéros tiraient la moyenne vers le bas.

Une décision non documentée est une décision cachée.

## Trois questions avant de passer à l'analyse

1. Combien de lignes ai-je perdues, et en quoi diffèrent-elles de celles que je garde ?
2. Chacune de mes décisions est-elle justifiable à voix haute devant quelqu'un qui conteste ?
3. Mon résultat principal survivrait-il à la décision inverse ?

La troisième est celle qui distingue un travail solide d'un travail plausible. C'est aussi une question type du contrôle continu.

**Pour la suite**

---

- Repérer les non-réponses codées en chiffres, et les convertir
- Distinguer non-réponse et hors champ — deux choses différentes
- Identifier une censure haute et dire ce qu'elle fait à un indice d'inégalité
- Documenter chaque décision, et mesurer son effet sur le résultat

Le livrable n'est pas un fichier propre : c'est le **journal des décisions**, avec l'effet chiffré de chacune.

1. Terminez votre journal de nettoyage sur `cm03-salaires.csv`.
2. Recalculez le salaire moyen après chaque étape et notez l'écart.
3. Reprenez `MOTIF_ABS` et `CONTRAT` : quels codes sont des non-réponses, lesquels sont du hors champ ?
4. Vérifiez que le notebook s'exécute de haut en bas.

`stefaniamarcassa.github.io/analyse_des_donnees`

**Des questions ?**